

Day 3, Session 2: Multi-Bit Registers

Concept Block (10 min)

What is a Register?

A register is simply multiple flip-flops working together in parallel, sharing the same clock.

Think of it this way:

- **1 flip-flop** = stores **1 bit**
- **8 flip-flops** = stores **8 bits** = **1 register (8-bit)**
- **16 flip-flops** = stores **16 bits** = **1 register (16-bit)**

Simple analogy:

- Flip-flop = **one parking spot** (holds one car)
- Register = **parking lot** (holds multiple cars, all managed by same security guard/clock)

Why Do Registers Exist?

In real digital systems, you **never** work with single bits alone:

- CPU stores 32-bit or 64-bit values
- Memory addresses are 16-bit or 32-bit
- Data buses carry 8, 16, or 32 bits simultaneously

Real-world examples:

1. **CPU registers:** EAX, EBX (32-bit) store computation results
2. **Program Counter (PC):** 32-bit register holds current instruction address
3. **Status Register:** 8-bit register holds CPU flags (zero, carry, overflow)
4. **Shift registers:** 8-bit registers in UART for serial communication
5. **Instruction Register:** Holds the current instruction being executed

Register vs Flip-Flop: What's Different?

Conceptually: Nothing! A register IS just multiple flip-flops.

In Verilog: The only difference is the bit-width declaration:

```
// Single flip-flop (1 bit)
output reg q;                // stores 1 bit

// 8-bit register
output reg [7:0] q;          // stores 8 bits

// 16-bit register
output reg [15:0] q;         // stores 16 bits
```

The always block logic is IDENTICAL:

```
// 1-bit flip-flop
always @(posedge clk or posedge rst) begin
    if (rst)
        q <= 1'b0;
    else
        q <= d;
end

// 8-bit register (SAME logic, wider bus)
always @(posedge clk or posedge rst) begin
    if (rst)
        q <= 8'b0;          // Clear all 8 bits
    else
        q <= d;              // Load all 8 bits at once
end
```

Key insight: Verilog handles the width automatically. You write the same logic, just with wider buses.

Understanding Bus Notation

This is critical. Master this NOW.

[7:0] → 8 bits (bit 7 is MSB, bit 0 is LSB)

[15:0] → 16 bits

[3:0] → 4 bits

[N-1:0] → N bits (parameterized)

Bit numbering:

Register [7:0]:

Bit:	7	6	5	4	3	2	1	0
	↑							↑
	MSB							LSB
	(Most Significant Bit)							(Least Significant Bit)

Example value: 8'b1010_1100

Bit 7 = 1 (MSB)

Bit 0 = 0 (LSB)

Decimal value = 172

Underscore in binary literals:

- `8'b1010_1100` = `8'b10101100`
- Underscore is for **readability only**, no effect on value
- Groups bits visually (like commas in numbers: 1,000,000)

Number format prefixes:

```
8'b10101100    // Binary (b)
8'hAC           // Hexadecimal (h) - same value
8'd172          // Decimal (d) - same value
8'o254          // Octal (o) - same value
```

All four represent the same 8-bit value!

Common Mistakes with Buses

✗ MISTAKE 1: Width mismatch

```
reg [7:0] q;
q <= 16'hFFFF; // WRONG: 16-bit value into 8-bit register
                // Synthesis truncates to 8'hFF (silent data loss!)
```

✗ MISTAKE 2: Wrong reset width

```
reg [7:0] q;
q <= 1'b0;      // WRONG: only clears 1 bit (others might be 'x')
q <= 8'b0;      // CORRECT: clears all 8 bits
q <= 8'd0;      // CORRECT: same as above
q <= 0;         // CORRECT: Verilog extends to match width
```

✗ MISTAKE 3: Reversed bit order

```
reg [0:7] q;    // WRONG convention (MSB at index 0)
reg [7:0] q;    // CORRECT convention (MSB at highest index)
```

Always use `[N-1:0]` format.

Code & Practical Block (12 min)

Design 1: Basic 8-bit Register

```
// reg8.v - Basic 8-bit Register
module reg8 (
    input wire    clk,      // Clock
    input wire    rst,      // Async reset (active HIGH)
    input wire [7:0] d,      // 8-bit data input
```

```

    output reg [7:0] q          // 8-bit data output
);

always @(posedge clk or posedge rst) begin
    if (rst)
        q <= 8'b0;           // Clear all 8 bits on reset
    else
        q <= d;               // Load all 8 bits simultaneously
    end

endmodule

```

This is literally a D flip-flop with 8-bit width. That's it.

Design 2: 8-bit Register with Enable

```

// reg8_en.v - 8-bit Register with Enable
module reg8_en (
    input wire      clk,        // Clock
    input wire      rst,        // Async reset
    input wire      en,         // Enable (load only when en=1)
    input wire [7:0] d,         // 8-bit data input
    output reg [7:0] q          // 8-bit data output
);

always @(posedge clk or posedge rst) begin
    if (rst)
        q <= 8'b0;             // Reset: clear all bits
    else if (en)
        q <= d;                 // Enabled: load new data
    else
        q <= q;                 // Disabled: hold current value
    end

endmodule

```

Same priority structure as D FF with enable:

- Reset > Enable > Hold
- Just wider buses (8-bit instead of 1-bit)

Design 3: Parameterized N-bit Register

// reg_param.v - Parameterized N-bit Register

```
module reg_param #(
    parameter WIDTH = 8          // Default width: 8 bits
) (
    input  wire          clk,
    input  wire          rst,
    input  wire          en,
    input  wire [WIDTH-1:0] d,    // Width depends on parameter
    output reg [WIDTH-1:0] q
);

    always @(posedge clk or posedge rst) begin
        if (rst)
            q <= {WIDTH{1'b0}};    // Clear all bits
        else if (en)
            q <= d;                // Load data
        else
            q <= q;                // Hold
    end

endmodule
```

Line-by-Line: Parameterized Register

Lines 2-3: Parameter declaration

```
module reg_param #()
```

```
    parameter WIDTH = 8
```

- `#( ... )` introduces **parameter list**
- `parameter WIDTH = 8` defines a parameter with **default value 8**
- Can be **overridden** when instantiating the module

Lines 7-8: Parameterized port widths

```
input  wire [WIDTH-1:0] d,
```

```
output reg  [WIDTH-1:0] q
```

- Width automatically adjusts based on `WIDTH` parameter
- If `WIDTH=8`: ports are `[7:0]`
- If `WIDTH=16`: ports are `[15:0]`

- If WIDTH=32: ports are [31:0]

Line 13: Parameterized reset

```
q <= {WIDTH{1'b0}};
```

- {WIDTH{1'b0}} is a **replication operator**
- Means: replicate 1'b0 exactly WIDTH times
- If WIDTH=8: becomes 8'b00000000
- If WIDTH=16: becomes 16'b0000000000000000
- Automatically matches register width!

Why not just q <= 0?

- q <= 0 also works (Verilog extends to match width)
- But {WIDTH{1'b0}} is **explicit** and **self-documenting**
- Industry practice: be explicit about widths

Instantiation: Using Parameters

```
// Instantiate 8-bit register (default)
```

```
reg_param uut_8bit (
    .clk(clk),
    .rst(rst),
    .en(en),
    .d(d_8),
    .q(q_8)
);
```

```
// Instantiate 16-bit register (override parameter)
```

```
reg_param #(.WIDTH(16)) uut_16bit (
    .clk(clk),
    .rst(rst),
    .en(en),
    .d(d_16),
    .q(q_16)
);
```

```
// Instantiate 32-bit register
```

```
reg_param #(.WIDTH(32)) uut_32bit (
    .clk(clk),
    .rst(rst),
```

```
.en(en),  
.d(d_32),  
.q(q_32)  
);
```

Key syntax: `#( .WIDTH(16) )` overrides the default parameter value.

Testbench: Testing All Three Designs

// reg_tb.v - Testbench for Registers

```
module reg_tb;  
    reg        clk;  
    reg        rst;  
    reg        en;  
    reg [7:0]  d;  
    wire [7:0] q_basic;  
    wire [7:0] q_en;  
    wire [7:0] q_param;  
  
    // Instantiate basic 8-bit register  
    reg8 uut_basic (  
        .clk(clk),  
        .rst(rst),  
        .d(d),  
        .q(q_basic)  
    );  
  
    // Instantiate 8-bit register with enable  
    reg8_en uut_en (  
        .clk(clk),  
        .rst(rst),  
        .en(en),  
        .d(d),  
        .q(q_en)  
    );  
  
    // Instantiate parameterized register (8-bit default)  
    reg_param #( .WIDTH(8) ) uut_param (  

```

```

        .clk(clk),
        .rst(rst),
        .en(en),
        .d(d),
        .q(q_param)
    );

// Clock generator
initial begin
    clk = 0;
    forever #5 clk = ~clk;
end

// Test stimulus
initial begin
    $dumpfile("reg_tb.vcd");
    $dumpvars(0, reg_tb);

    $display("Time | rst | en | d          | q_basic | q_en    | q_param");

    $display("-----");
    $monitor("%4t | %b | %b | %b | %b | %b | %b",
            $time, rst, en, d, q_basic, q_en, q_param);

    // Initialize
    rst = 1; en = 0; d = 8'h00;
    #10;
    rst = 0;
    #10;

    // Test 1: Load data (basic register always loads)
    $display("\n--- Test 1: Load Data ---");
    d = 8'hA5; #10;    // 8'hA5 = 8'b10100101
    d = 8'h3C; #10;    // 8'h3C = 8'b00111100
    d = 8'hFF; #10;    // All 1s
    d = 8'h00; #10;    // All 0s

```

```

// Test 2: Enable control (only q_en and q_param affected)
$display("\n--- Test 2: Enable=1, Load ---");
en = 1;
d = 8'hAA; #10;    // 8'hAA = 8'b10101010

$display("\n--- Test 3: Enable=0, Hold ---");
en = 0;
d = 8'h55; #10;    // q_basic loads 0x55, but q_en and q_param HOLD 0xAA
d = 8'hFF; #10;    // q_basic loads 0xFF, but q_en and q_param still
HOLD

// Test 4: Re-enable
$display("\n--- Test 4: Re-enable ---");
en = 1;
d = 8'h42; #10;    // All registers load 0x42

// Test 5: Reset during operation
$display("\n--- Test 5: Async Reset ---");
d = 8'hFF; #5;     // Mid-cycle
rst = 1; #10;      // All registers clear to 0x00
rst = 0; #10;

$finish;
end

endmodule

```

Compile and Test

```

iverilog -o sim/reg_tb reg8.v reg8_en.v reg_param.v reg_tb.v
vvp sim/reg_tb
gtkwave reg_tb.vcd

```

Expected Behavior

Key differences between the three registers:

Register	Loads when en=0?	Loads when en=1?
reg8 (basic)	✓ Always loads	✓ Always loads

Register	Loads when en=0?	Loads when en=1?
reg8_en (with enable)	 Holds	 Loads
reg_param (parameterized)	 Holds	 Loads

In GTKWave:

- When en=0: q_basic changes with d, but q_en and q_param stay constant
- When en=1: All three registers load data
- When rst=1: All three immediately clear to 8'h00

Mini Task (5-8 min)

Task: Instantiate **three different width registers** using reg_param and verify they all work.

Requirements:

1. Create a testbench that instantiates:
 - 4-bit register (WIDTH=4)
 - 8-bit register (WIDTH=8)
 - 16-bit register (WIDTH=16)
2. Load different test values into each
3. Verify enable and reset work for all widths

Module signature:

```
module reg_multi_tb;
    reg clk, rst, en;

    // Declare buses of different widths
    reg [3:0] d_4;
    reg [7:0] d_8;
    reg [15:0] d_16;

    wire [3:0] q_4;
    wire [7:0] q_8;
    wire [15:0] q_16;

    // Instantiate three reg_param with different widths
    // TODO: Your code here

    // Clock, stimulus, monitor
    // TODO: Your code here
endmodule
```

Hint for instantiation:

```
reg_param #(.WIDTH(4)) reg_4bit (  
    .clk(clk),  
    .rst(rst),  
    .en(en),  
    .d(d_4),  
    .q(q_4)  
);  
// Do same for 8-bit and 16-bit
```

Submit your testbench. Show me the simulation output when it runs.

Once verified, we move to **Shift Registers** (Day 3, Session 3).